Finding the Minimum and Inserting in Place

Two More O(n^2) Strategies

Lecture 22

Data Structures and Algorithms

Where We Left Off

Last lecture, we introduced the world of sorting and studied **Bubble Sort** -- the simplest comparison-based sorting algorithm.

We also learned that every sorting algorithm can be described by three key properties:

1. **Stability** -- Does it preserve the relative order of equal elements?
2. **In-place** -- Does it use only O(1) extra memory?
3. **Adaptive** -- Does it run faster on nearly-sorted input?

Bubble Sort is stable, in-place, and adaptive (with the early-termination flag), but its $O(n^2)$ worst case is abysmal for large data.

Today we meet two more simple sorts. One minimizes the number of swaps. The other is the best simple sort in practice.

Part 1: Selection Sort

"Find the Smallest, Put It in Place"

The Selection Sort Idea

Selection Sort takes a beautifully simple approach:

1. Divide the array into two regions: a **sorted region** on the left and an **unsorted region** on the right.
2. In each pass, **find the minimum element** in the unsorted region.
3. **Swap** it with the first element of the unsorted region.
4. The sorted region grows by one. Repeat.

Analogy: Imagine you are lining up students by height. You scan the entire line, find the shortest student, and bring them to the front. Then you scan the remaining students, find the next shortest, and place them second. And so on.

Unlike Bubble Sort, which makes many swaps per pass, Selection Sort makes **exactly one swap per pass**.

Selection Sort: The Algorithm

```
SelectionSort(A, n):
    for i = 0 to n-2:
        minIndex = i
        for j = i+1 to n-1:
            if A[j] < A[minIndex]:
                minIndex = j
        swap(A[i], A[minIndex])
```

What each line does:

- **Outer loop (`i`):** Selects the position to fill next. After iteration `i`, the first `i+1` elements are sorted.
- **Inner loop (`j`):** Scans the unsorted portion to find the minimum element.
- **`minIndex`:** Tracks the index of the smallest element found so far.
- **Swap:** Places the minimum into its correct sorted position. Only **one swap per pass** -- no matter how large the array is.

Selection Sort: Complete Trace

Array: [29, 10, 14, 37, 13]

Pass 1 (i=0): Find minimum in the entire array [29, 10, 14, 37, 13]

```
Scan: 29, 10, 14, 37, 13
                              min = 10 at index 1

Swap A[0] and A[1]:  [10, | 29, 14, 37, 13]
                      ^^^^
                     Sorted
```

The smallest element (10) is now at the front.

Selection Sort: Passes 2 through 4

Pass 2 (i=1): Find min in [29, 14, 37, 13]

```
min = 13 at index 4.   Swap A[1] and A[4].

[10, 13, | 14, 37, 29]
^^^^^^^^^^
  Sorted
```

Pass 3 (i=2): Find min in [14, 37, 29]

```
min = 14 at index 2.   Already in the correct position! Swap A[2] with A[2] (no-op).

[10, 13, 14, | 37, 29]
^^^^^^^^^^^^^^
    Sorted
```

Pass 4 (i=3): Find min in [37, 29]

```
min = 29 at index 4.   Swap A[3] and A[4].

[10, 13, 14, 29, | 37]
^^^^^^^^^^^^^^^^^^
     Sorted           --> Done! [10, 13, 14, 29, 37]
```

Selection Sort: The Sorted Boundary

Visualizing the growing sorted region:

```
Start:    [29, 10, 14, 37, 13]          Unsorted: all 5
Pass 1:   |10| 29, 14, 37, 13           Unsorted: 4
Pass 2:   |10, 13| 14, 37, 29           Unsorted: 3
Pass 3:   |10, 13, 14| 37, 29           Unsorted: 2
Pass 4:   |10, 13, 14, 29| 37           Unsorted: 1
Done:     |10, 13, 14, 29, 37|          Sorted!
```

Compare this with Bubble Sort: Bubble Sort builds the sorted region on the **right** (largest elements settle first). Selection Sort builds it on the **left** (smallest elements are placed first).

Selection Sort: Complexity Analysis

Case	Comparisons	Swaps	Time
Best case	$\frac{n(n-1)}{2}$	$n-1$	$O(n^2)$
Average case	$\frac{n(n-1)}{2}$	$n-1$	$O(n^2)$
Worst case	$\frac{n(n-1)}{2}$	$n-1$	$O(n^2)$
Space	--	--	$O(1)$

Key observations:

- The number of **comparisons is always the same** -- $\frac{n(n-1)}{2}$ -- regardless of input order. Selection Sort has no best case; it always scans the entire unsorted portion.
- The number of **swaps is always exactly $n-1$** -- the minimum among all simple sorts. This makes it ideal when writes are expensive (e.g., writing to flash memory or EEPROM).
- **Not adaptive:** A sorted array takes just as long as a reverse-sorted array.
- **In-place:** Yes.

Selection Sort is UNSTABLE

This is a critical property that distinguishes Selection Sort from Bubble Sort.

Example: Sort [5a, 5b, 3] (subscripts track original order of equal elements)

```
Pass 1: Find min in [5a, 5b, 3].  min = 3 at index 2.

        Swap A[0] and A[2]:  [3, 5b, 5a]
                                  ^^  ^^
                            5b is now BEFORE 5a!
                            Original order was 5a before 5b.
```

Result: [3, 5b, 5a] -- the relative order of equal elements (5a and 5b) has been **reversed**.

Why does this happen? The long-range swap jumps 5a over 5b without regard for their relative order. This is fundamentally different from Bubble Sort, which only swaps adjacent elements and therefore never disturbs the order of equal items.

Part 2: Insertion Sort

"The Card Player's Algorithm"

The Insertion Sort Idea

Imagine you are holding a hand of playing cards, sorted from left to right. A new card is dealt to you. You pick it up and slide it into its correct position among the cards you already hold.

This is exactly how Insertion Sort works:

1. Start with the first element -- a "hand" of one card is trivially sorted.
2. Take the next element (the "key") from the unsorted region.
3. Compare it with each element in the sorted region, moving from right to left.
4. **Shift** elements rightward to make room, then **insert** the key in its correct position.
5. Repeat until the entire array is sorted.

The sorted region grows from the left, one element at a time -- just like picking up and inserting cards one by one.

Insertion Sort: The Algorithm

```
InsertionSort(A, n):
    for i = 1 to n-1:
        key = A[i]
        j = i - 1
        while j >= 0 and A[j] > key:
            A[j+1] = A[j]        // shift element right
            j = j - 1
        A[j+1] = key              // insert key in correct position
```

What each line does:

- **Outer loop (`i`):** Picks the next unsorted element as the "key."
- **`key = A[i]`:** Saves the element to be inserted (so it is not lost during shifting).
- **Inner while loop:** Walks backward through the sorted region, shifting elements right to make room.
- **`A[j+1] = key`:** Drops the key into the gap created by the shifts.

Notice: There are no explicit swaps. Insertion Sort uses **shifts** (one-directional moves), not swaps. This is more efficient and is what preserves stability.

Insertion Sort: Complete Trace

Array: [12, 11, 13, 5, 6]

i = 1: key = 11. Sorted region: [12]

```
Compare 11 with 12:  12 > 11, shift 12 right.
Insert 11 at position 0.

[11, 12, | 13, 5, 6]
^^^^^^^^^^
  Sorted
```

i = 2: key = 13. Sorted region: [11, 12]

```
Compare 13 with 12:  12 < 13, STOP.  (13 is already in the right place)
No shifts needed. Insert 13 in place.

[11, 12, 13, | 5, 6]
^^^^^^^^^^^^^^
    Sorted
```

Insertion Sort: Passes 3 and 4

i = 3: key = 5. Sorted region: [11, 12, 13]

```
Compare 5 with 13:  13 > 5, shift 13 right.    [11, 12, __, 13, 6]
Compare 5 with 12:  12 > 5, shift 12 right.    [11, __, 12, 13, 6]
Compare 5 with 11:  11 > 5, shift 11 right.    [__, 11, 12, 13, 6]
j = -1, STOP.  Insert 5 at position 0.

[5, 11, 12, 13, | 6]
^^^^^^^^^^^^^^^^^
     Sorted
```

i = 4: key = 6. Sorted region: [5, 11, 12, 13]

```
Compare 6 with 13:  shift.    [5, 11, 12, __, 13]
Compare 6 with 12:  shift.    [5, 11, __, 12, 13]
Compare 6 with 11:  shift.    [5, __, 11, 12, 13]
Compare 6 with 5:   5 < 6, STOP.  Insert 6 at position 1.

[5, 6, 11, 12, 13]
^^^^^^^^^^^^^^^^^^  --> Done! Sorted!
```

Insertion Sort: The Sorted Boundary

```
Start:    [12, 11, 13, 5, 6]
i = 1:    |11, 12| 13, 5, 6          key=11 inserted before 12
i = 2:    |11, 12, 13| 5, 6          key=13 stayed in place
i = 3:    |5, 11, 12, 13| 6          key=5 traveled all the way to front
i = 4:    |5, 6, 11, 12, 13|         key=6 inserted after 5

Done:     [5, 6, 11, 12, 13]
```

Notice how different keys travel different distances during insertion. Key 13 traveled zero positions (already in place), while key 5 traveled three positions to the front. This is what makes Insertion Sort **adaptive** -- if the array is nearly sorted, most keys barely move.

Insertion Sort: Complexity Analysis

Case	Comparisons	Shifts	Time
Best case (already sorted)	$n-1$	0	$O(n)$
Average case	$\frac{n(n-1)}{4}$	$\frac{n(n-1)}{4}$	$O(n^2)$
Worst case (reverse sorted)	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$O(n^2)$
Space	--	--	$O(1)$

Properties:

- **In-place:** Yes -- only one temp variable (`key`).
- **Stable:** Yes -- we use `>` (strict), so equal elements are never reordered. The key is inserted *after* any equal elements in the sorted region.
- **Adaptive:** Extremely. On nearly-sorted data, the inner loop barely executes, making performance approach O(n).

Why Insertion Sort is Special

Among the three simple sorts, Insertion Sort is the only one that sees serious use in real-world software:

1. Best for small arrays

Python's Timsort and Java's `Arrays.sort` switch to Insertion Sort for subarrays of size 16-32. The overhead of divide-and-conquer algorithms is not worth it for tiny arrays.

2. Best for nearly-sorted data

If an array is "almost sorted" (only a few elements are out of place), Insertion Sort runs in nearly O(n) time. No other simple sort offers this.

3. Online sorting

Insertion Sort can sort data **as it arrives** -- each new element is inserted into the already-sorted portion without restarting from scratch. Ideal for streaming data.

As Reema Thareja puts it: "*Insertion sort is the algorithm of choice when the input is nearly sorted or when the dataset is small.*"

Verifying Insertion Sort's Stability

Let us trace the same test case we used for Selection Sort: **[5a, 5b, 3]**

```
i = 1: key = 5b.  Sorted: [5a]
       Compare 5b with 5a:  5a > 5b?  NO (they are equal, and we use strict >).
       STOP.  Insert 5b after 5a.
       Array: [5a, 5b, 3]

i = 2: key = 3.   Sorted: [5a, 5b]
       Compare 3 with 5b:  5b > 3, shift.   [5a, __, 5b]
       Compare 3 with 5a:  5a > 3, shift.   [__, 5a, 5b]
       Insert 3 at position 0.
       Array: [3, 5a, 5b]
```

Result: [3, 5a, 5b] -- the two 5s remain in their original order. Insertion Sort is stable because equal elements are never shifted past each other.

Compare: Selection Sort produced [3, 5b, 5a] -- unstable!

Part 3: Head-to-Head Comparison

The Simple Sorts: All in One Table

Feature	Bubble Sort	Selection Sort	Insertion Sort
Best Case	$O(n)$	$O(n^2)$	$O(n)$
Average Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Worst Case	$O(n^2)$	$O(n^2)$	$O(n^2)$
Space	$O(1)$	$O(1)$	$O(1)$
Stable?	Yes	No	Yes
In-Place?	Yes	Yes	Yes
Swaps (worst)	$\frac{n(n-1)}{2}$	$n-1$	0 (shifts only)
Comparisons (worst)	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$	$\frac{n(n-1)}{2}$
Adaptive?	Yes (with flag)	No	Yes

All three are $O(n^2)$ in the worst case. All three are in-place. But they have very different strengths.

When to Use Which?

Bubble Sort

- **Almost never in practice.** Its only advantage is conceptual simplicity.
- Useful for teaching and for verifying that a small array is already sorted (one O(n) pass with the flag).

Selection Sort

- When **memory writes are expensive** -- for example, writing to flash memory, EEPROM, or other storage where writes cause wear.
- Selection Sort guarantees exactly $n - 1$ swaps, the minimum among all simple sorts.
- It does not matter that it does many comparisons -- comparisons are cheap (reads), swaps are expensive (writes).

Insertion Sort

- **The practical winner among simple sorts.**
- Best for **small arrays** ($n \leq 30$) -- used as a subroutine in Timsort, Introsort, and other hybrid algorithms.
- Best for **nearly sorted data** -- approaches O(n) performance.
- Best for **online sorting** -- processes elements as they arrive.

A Concrete Comparison: Sorting [5, 3, 8, 6, 2]

Bubble Sort (optimized):

```
Pass 1: [3,5,6,2,8]  -- 3 swaps, 8 settled
Pass 2: [3,5,2,6,8]  -- 1 swap,  6 settled
Pass 3: [3,2,5,6,8]  -- 1 swap,  5 settled
Pass 4: [2,3,5,6,8]  -- 1 swap,  3 settled
Total:  4 passes, 6 swaps
```

Selection Sort:

```
Pass 1: min=2, swap A[0]<->A[4]: [2,3,8,6,5]   -- 1 swap
Pass 2: min=3, already in place: [2,3,8,6,5]    -- 1 swap (self)
Pass 3: min=5, swap A[2]<->A[4]: [2,3,5,6,8]   -- 1 swap
Pass 4: min=6, already in place: [2,3,5,6,8]    -- 1 swap (self)
Total:  4 passes, 4 swaps (always exactly n-1)
```

Insertion Sort:

```
i=1: key=3, shift 5 right, insert 3: [3,5,8,6,2]  -- 1 shift
i=2: key=8, no shift:                [3,5,8,6,2]  -- 0 shifts
i=3: key=6, shift 8 right, insert 6: [3,5,6,8,2]  -- 1 shift
i=4: key=2, shift 8,6,5,3, insert 2: [2,3,5,6,8]  -- 4 shifts
Total:  4 passes, 6 shifts (0 swaps!)
```

Why Can't We Do Better Than O(n^2)?

All three simple sorts are $O(n^2)$ in the worst case. A natural question arises:

Is there a fundamental limit on how fast comparison-based sorting can be?

The answer is **yes** -- and the limit is $\Omega(n \log n)$, not $O(n^2)$.

There exist algorithms (Merge Sort, Quicksort, Heap Sort) that achieve $O(n \log n)$. This means our simple sorts are **not optimal** -- they do roughly $n^2/2$ comparisons when only about $n \log n$ are mathematically necessary.

How do we prove this lower bound? Through a beautiful argument involving **decision trees**. Next lecture, we will prove that $\Omega(n \log n)$ is the speed limit of sorting, and then we will meet **Merge Sort** -- the first algorithm that hits that limit, guaranteed.

Part 4: Summary and What is Next

Today's Key Concepts

Selection Sort:

- Find the minimum, swap it to the front. Repeat.
- Always $O(n^2)$ -- no best case optimization possible.
- Exactly $n - 1$ swaps -- fewest writes of any simple sort.
- **Unstable** -- the long-range swap can reorder equal elements.

Insertion Sort:

- Pick the next element, insert it into the sorted region by shifting.
- $O(n)$ best case on sorted input, $O(n^2)$ worst case.
- Uses shifts (not swaps) -- more efficient and preserves stability.
- **The best simple sort in practice** -- used in real-world hybrid algorithms.

The Big Picture:

- All simple sorts are $O(n^2)$ worst case, but each has a unique strength.
- The $O(n^2)$ barrier is not the best we can do -- $O(n \log n)$ is achievable.

Complexity Summary So Far

Algorithm	Best	Average	Worst	Space	Stable?	In-Place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes

We will add Merge Sort and Quicksort to this table in the coming lectures. By the end of the unit, you will have a complete guide for choosing the right algorithm for any scenario.

Visual Tool: See All Three Sorts Side by Side

VisuAlgo -- Sorting Algorithm Visualizer:
https://visualgo.net/en/sorting

Select Bubble Sort, Selection Sort, or Insertion Sort from the menu. Watch how each one builds the sorted region differently:

- Bubble Sort pushes the **largest** to the right each pass
- Selection Sort places the **smallest** on the left each pass
- Insertion Sort **inserts** each element into its correct place in the growing sorted region

Try sorting the **same array** with all three algorithms to compare the number of operations visually.

Review Questions

1. Given [7, 3, 5, 3, 2], trace both Selection Sort and Insertion Sort. Which one preserves the relative order of the two 3s? Verify your answer.
2. Selection Sort always makes exactly $n - 1$ swaps. Explain why this is true regardless of the input order.
3. Consider a nearly-sorted array where only the last element is out of place: [1, 2, 3, 4, 5, 6, 7, 0]. How many comparisons does Insertion Sort make? How many does Selection Sort make? What does this tell you about adaptivity?
4. A hardware engineer needs to sort sensor readings stored in flash memory, where each write degrades the memory. Which simple sort should they use, and why?
5. Can you modify Selection Sort to make it stable? What would you have to change, and what trade-off would it introduce? (Hint: think about how Insertion Sort handles placement.)

What Comes Next

Lecture 23 tackles one of the most elegant results in computer science: the $\Omega(n \log n)$ **lower bound for comparison-based sorting**.

We will prove, using the **decision tree model**, that no comparison-based sorting algorithm can do better than $n \log n$ comparisons in the worst case. This is a fundamental speed limit -- as absolute as the speed of light in physics.

Then we will meet the algorithm that achieves this limit: **Merge Sort** -- a beautiful divide-and-conquer algorithm invented by John von Neumann in 1945. It splits the array in half, recursively sorts each half, and merges them back together in $O(n)$ time.

Merge Sort guarantees $O(n \log n)$ in all cases. No exceptions. No worst-case surprises. The era of $O(n^2)$ is over.